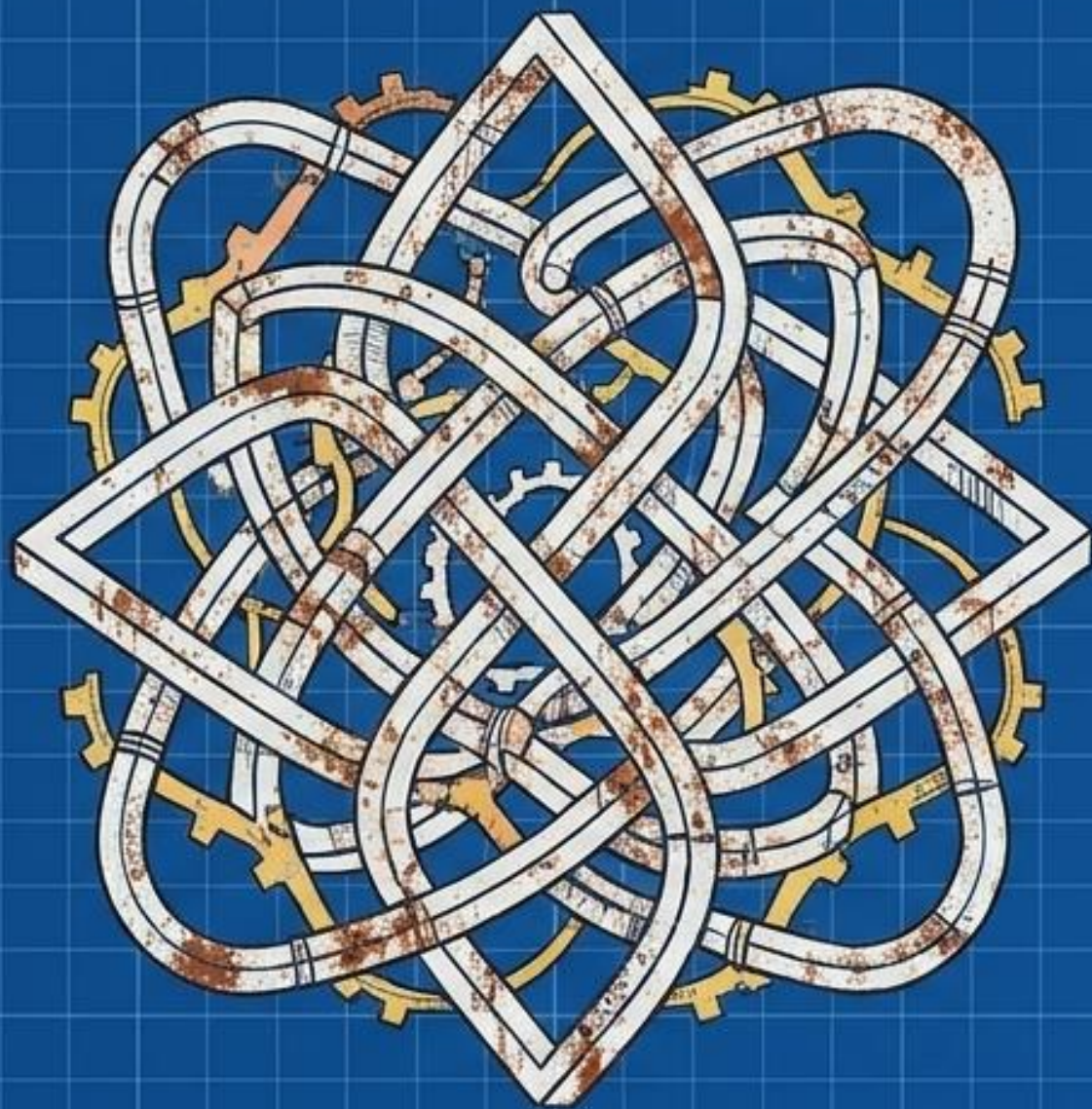
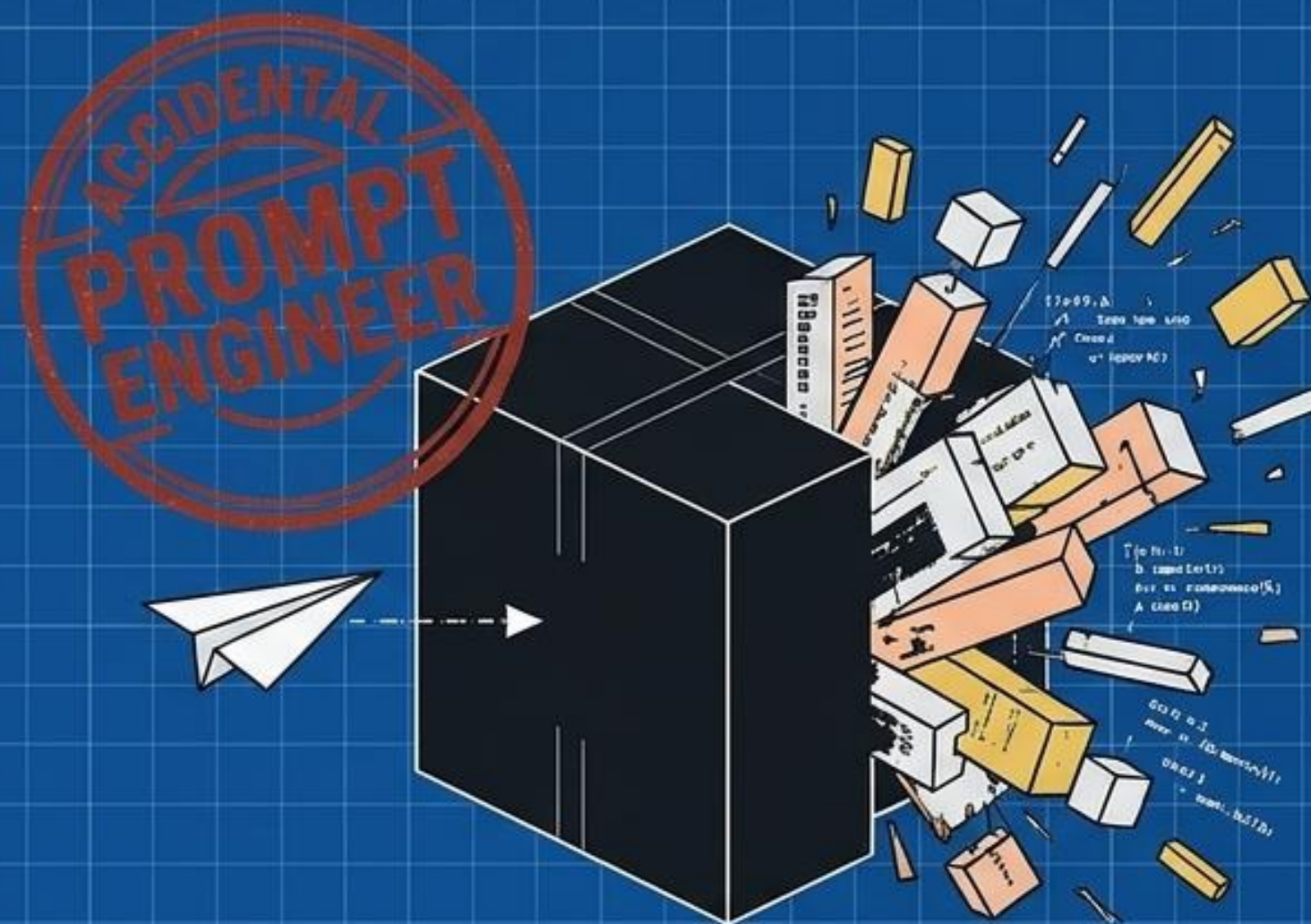


El Laberinto Tradicional



Centrado en el Código. Los requisitos caducan, la documentación se desactualiza y los bugs se esconden en sistemas incomprensibles.

La Trampa del Prompt & Pray



"VIP Coding" con IA. Generar código rápido sin arreglar el proceso subyacente solo acelera el caos. La IA no sabe qué estás construyendo.

El Cambio de Paradigma: Spec-Driven Development



El Cambio de Paradigma. El código ya no es el centro. La IA genera, regenera y refactoriza los detalles de implementación alrededor de las especificaciones estables.

Matriz de Evolución del Software

	Tradicional	Prompt & Pray (IA Naïve)	Spec-Driven (AIUP)
Fuente de Verdad	Código fuente	El último prompt	La Especificación
Rol de la IA	Inexistente	Generador mágico ciego	Motor de consistencia
Rol Humano	Escribir sintaxis	Adivinar prompts	Diseñar intención y arquitectura
Mantenibilidad	Se degrada con el tiempo	Deuda técnica instantánea	Evolución sostenible



AI Unified Process (AIUP) y sus Pilares

Software AI-Native

Centrado en Especificaciones

La intención clara y las especificaciones guían a la IA para generar código significativo.

Consciente del Contexto

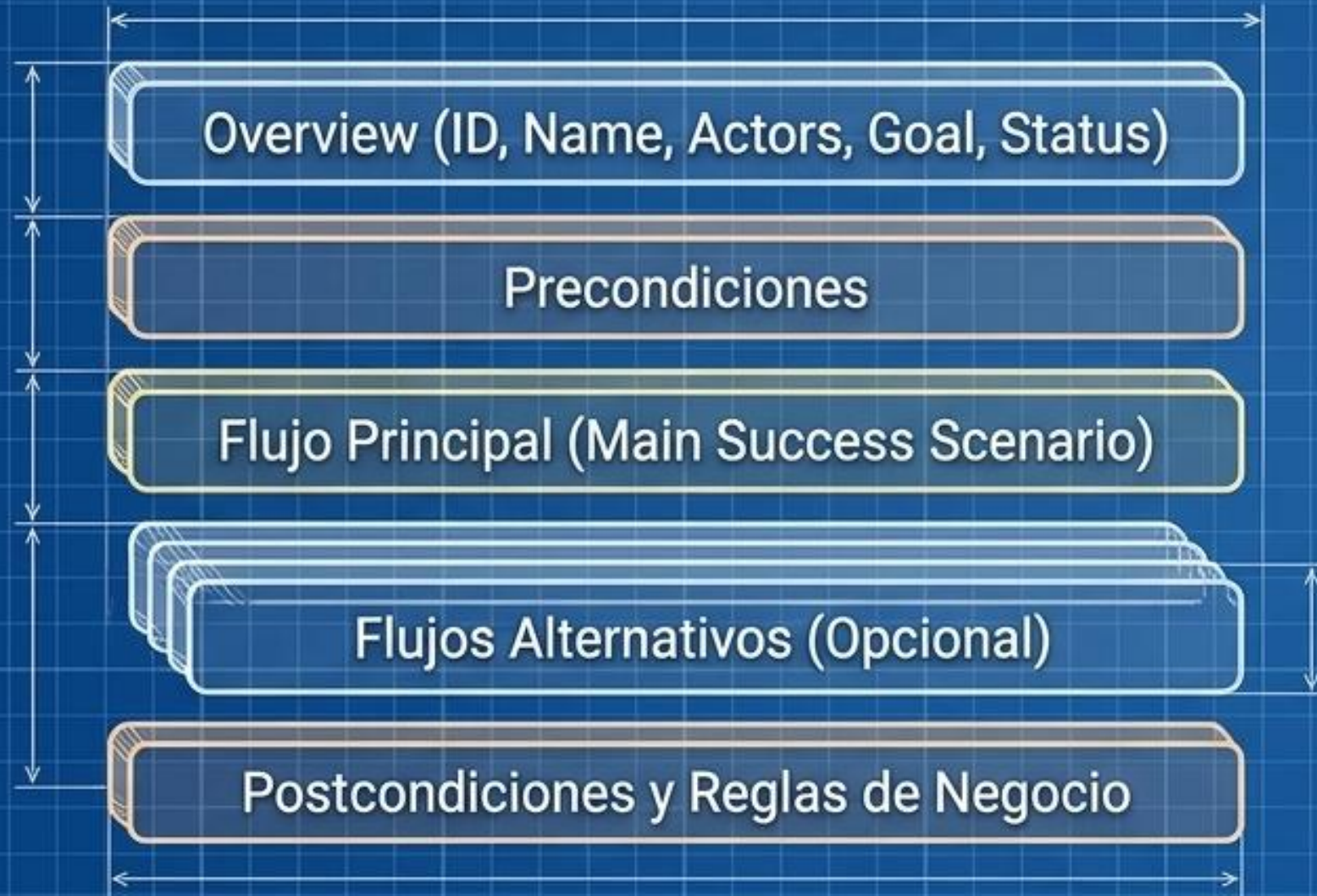
Los agentes de IA comprenden el contexto completo del sistema y sus guardarrailes.

Experiencia del Agente

Tareas de IA autónomas, servidores MCP y herramientas diseñadas para el rendimiento de la IA, no solo del humano.

Casos de Uso Estructurados vs. Prompts Simples

El Estándar: Caso de Uso Estructurado

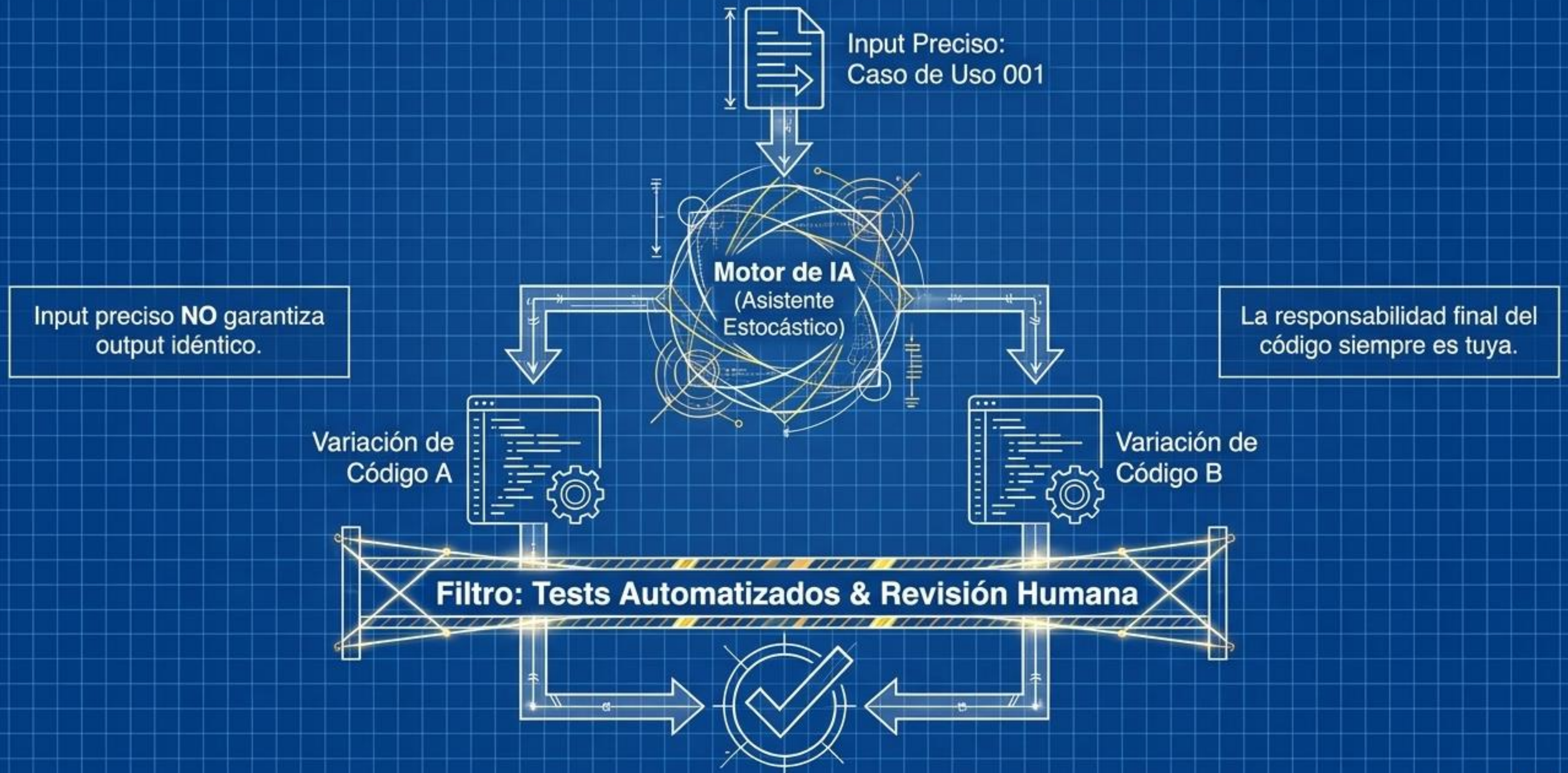


El Error: Prompt Naïve



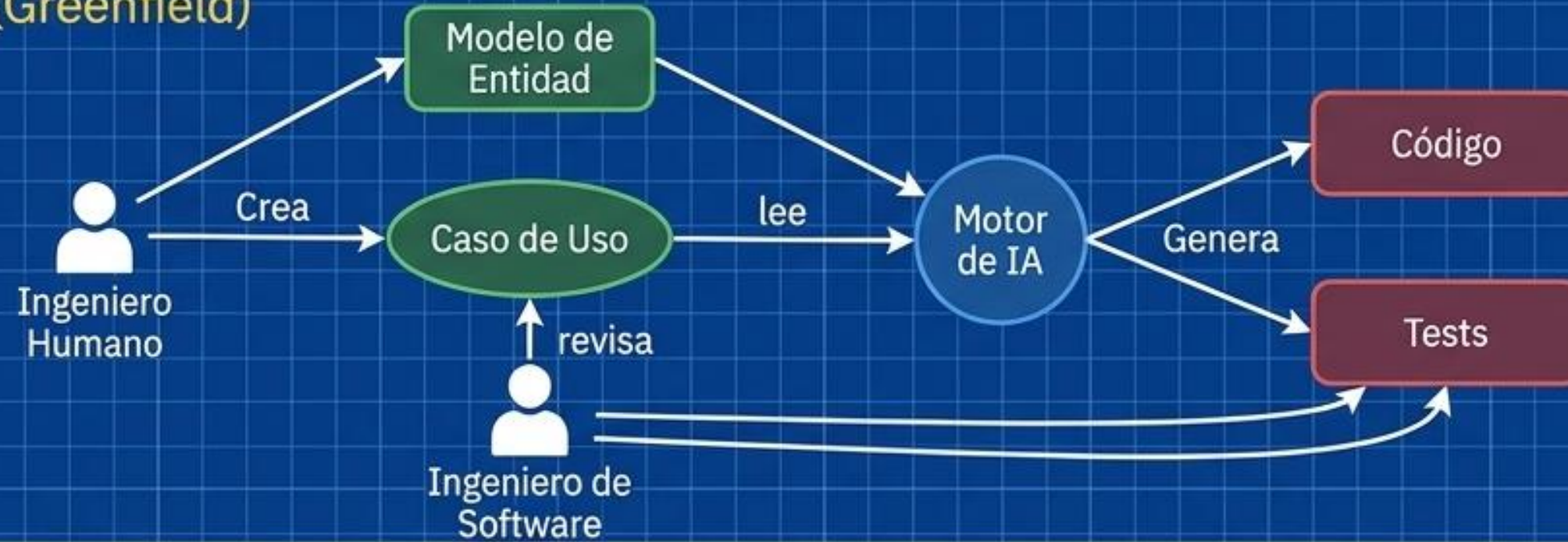
Un prompt es un deseo. Un Caso de Uso es un contrato estructurado. Define flujos completos y reglas de negocio, permitiendo un tamaño de entrega exacto.

La IA NO es un Compilador

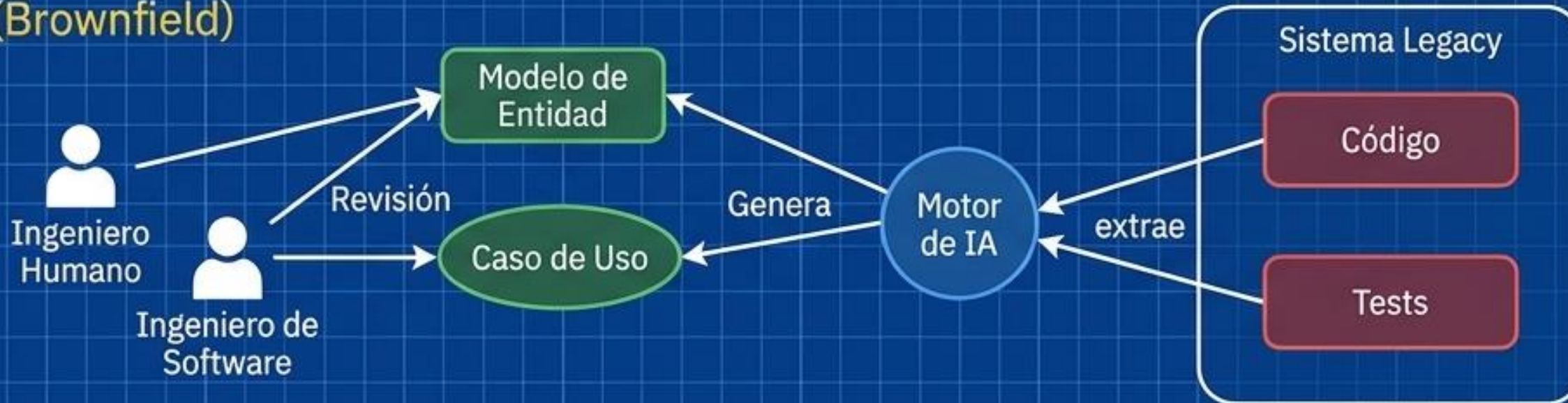


Flujos de Trabajo SDD: Greenfield vs. Brownfield

Proyectos Nuevos (Greenfield)



Modernización (Brownfield)



Brownfield es ingeniería inversa de especificaciones. Las especificaciones salvan a las empresas de su propio código legacy.

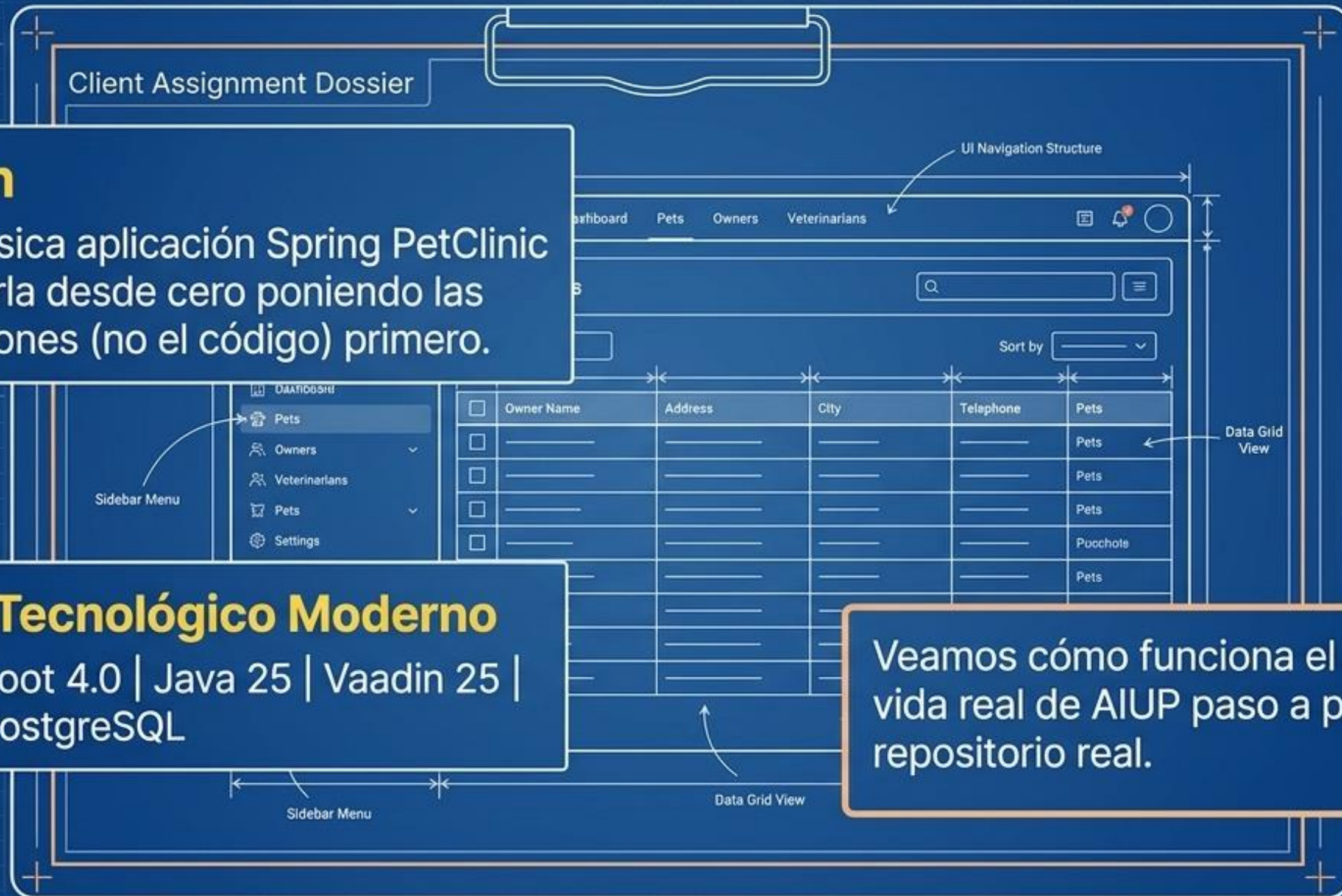
Caso Práctico: Reconstruyendo PetClinic

La Misión

Tomar la clásica aplicación Spring PetClinic y reconstruirla desde cero poniendo las especificaciones (no el código) primero.

Stack Tecnológico Moderno

Spring Boot 4.0 | Java 25 | Vaadin 25 |
jOOQ | PostgreSQL



Veamos cómo funciona el ciclo de vida real de AIUP paso a paso en un repositorio real.

Paso 1: Guardarraíles y Modelo de Dominio

Responsabilidad Humana

El humano establece la arquitectura, la seguridad y los límites tecnológicos.



Responsabilidad IA

El Agente extrae el Modelo de Dominio a partir del entendimiento del negocio.



Paso 2: Implementación Guiada por Especificaciones

La Especificación Fija

File: docs/use_cases/UC-001.md

UC-001: Welcome Page

****Actor:**** Usuario

****Flujo Principal:****

1. El sistema carga el layout principal.
2. Muestra un menú de navegación.
3. Renderiza la imagen de bienvenida de PetClinic.

La Ejecución del Agente

> /implement use case 001

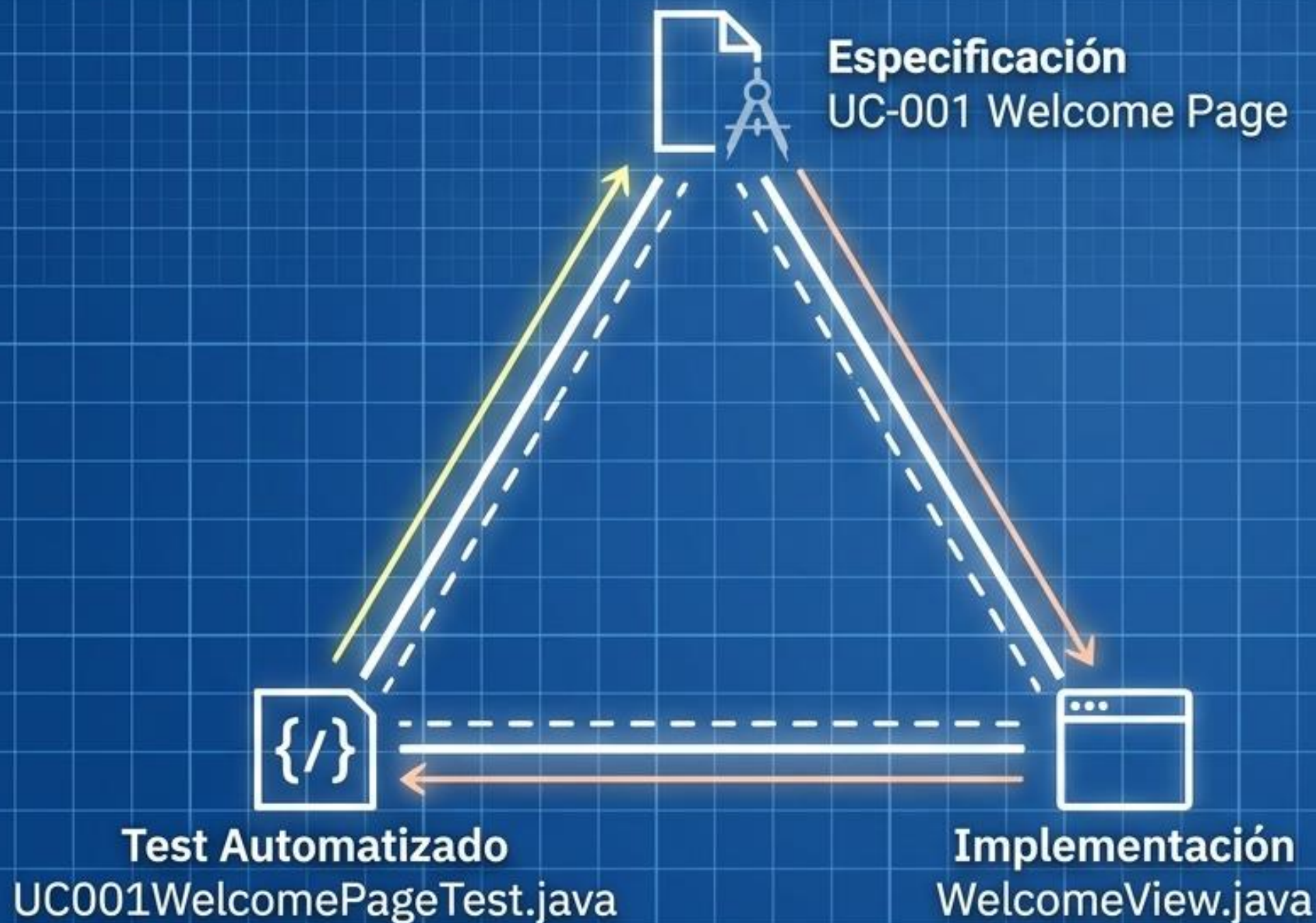
[OK] Reading UC-001.md

[OK] Analyzing architecture guardrails...

[OK] Generating Vaadin Views...

Cero 'Prompting' Manual. El agente de IA ejecuta una habilidad (skill) predefinida contra el Caso de Uso formal. El agente lee el repositorio, los **guardarrailes** y la especificación de forma autónoma.

Paso 3: Tests y Consistencia 1:1:1

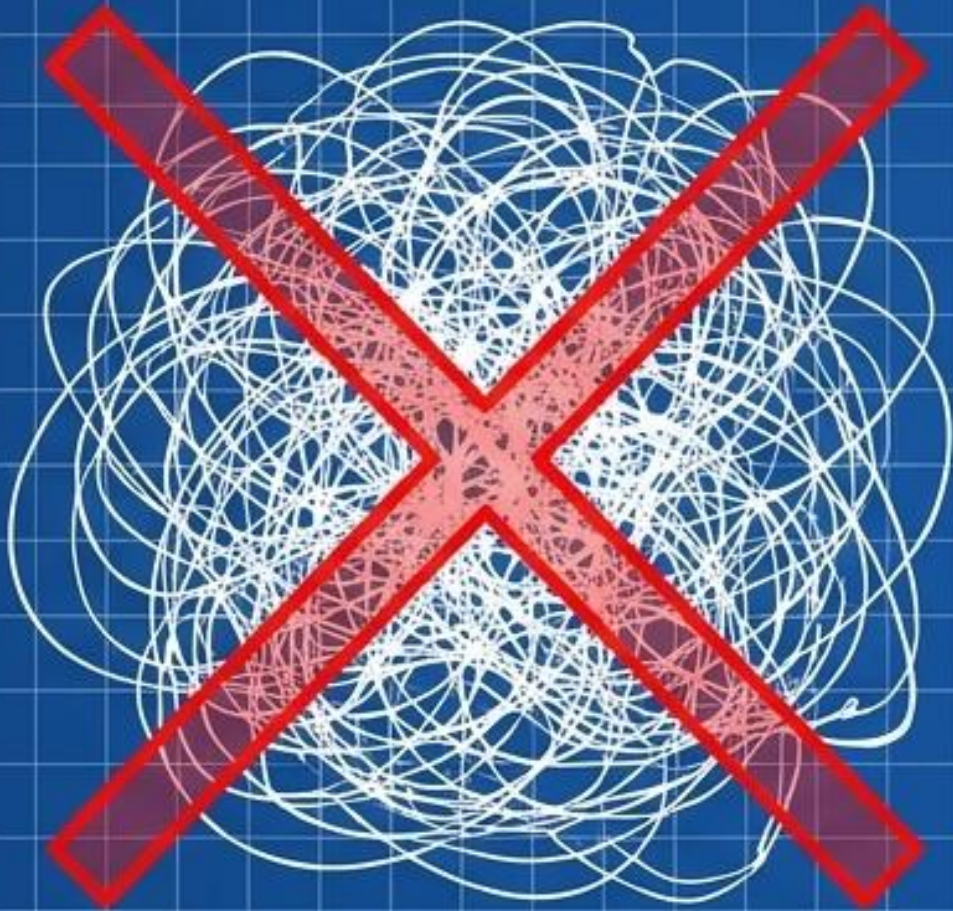


Test-Driven Consistency:
Las pruebas demuestran que la IA cumplió el contrato.

Si la IA regenera el código mañana de forma diferente, los tests garantizan que el comportamiento del negocio sea idéntico.

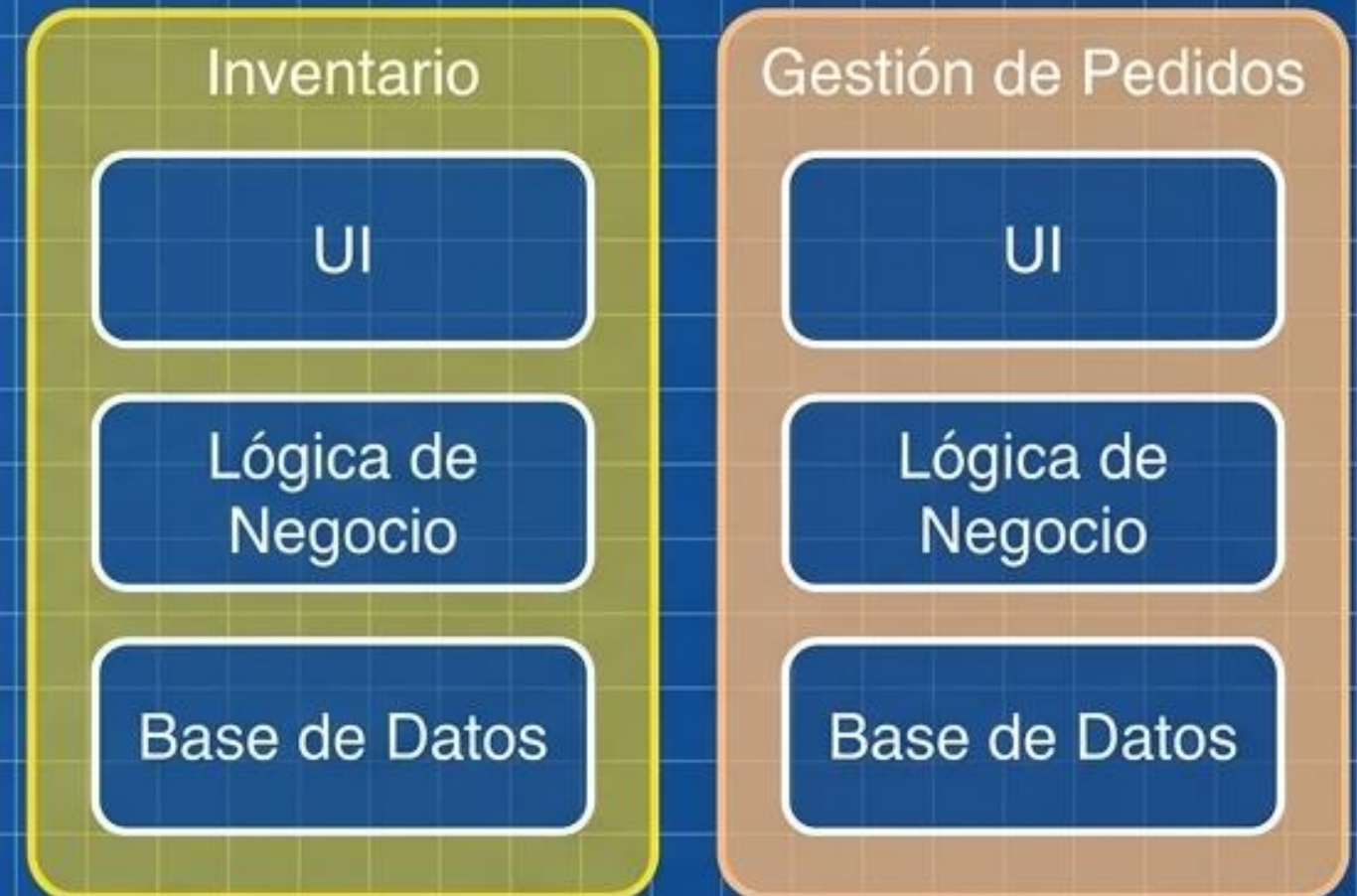
El Papel de la Arquitectura

Contexto Infinito (Monolito)



La IA alucina. Los humanos se pierden en el caos.

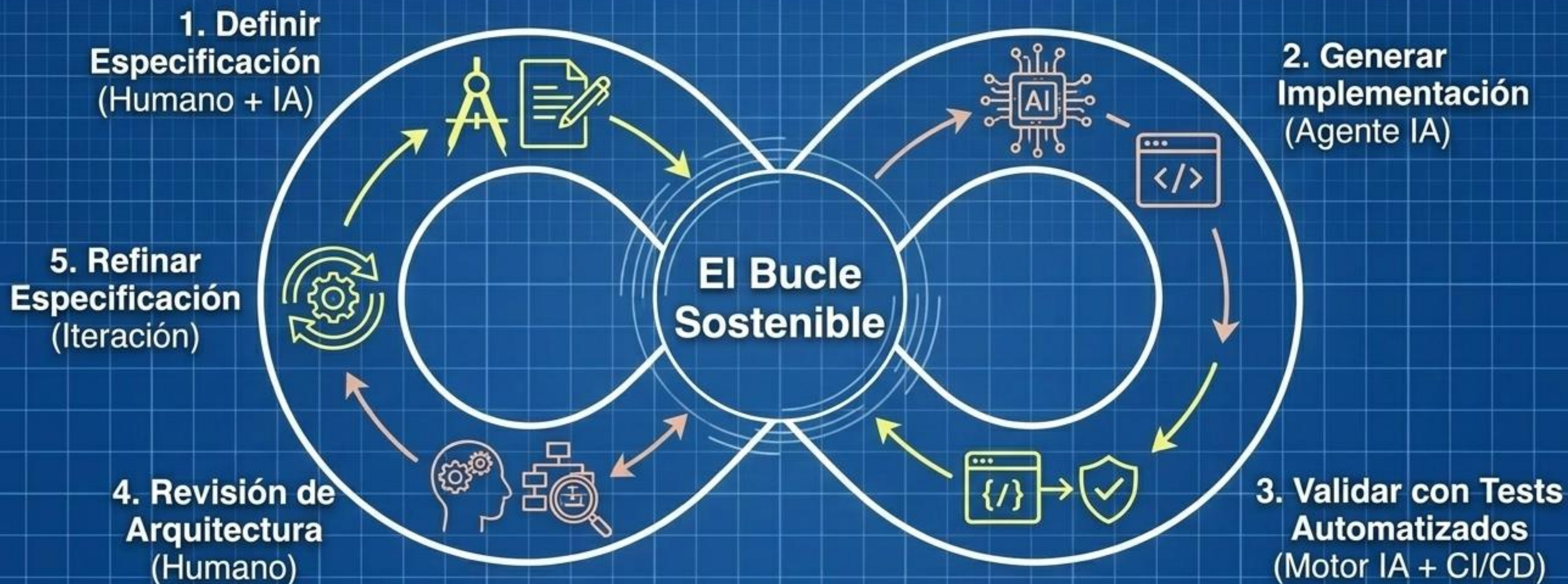
Sistemas Autocontenidos (SCS)



Tamaño de contexto reducido. Precisión máxima.

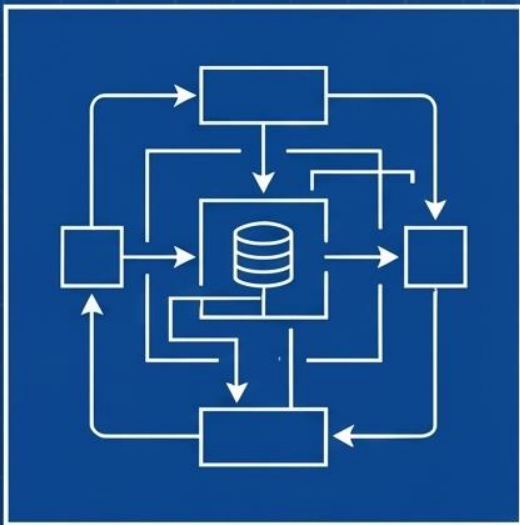
Diseños modulares limitan el tamaño del contexto, haciendo que la generación de la IA sea increíblemente precisa y segura.

El Ciclo de Vida SDD



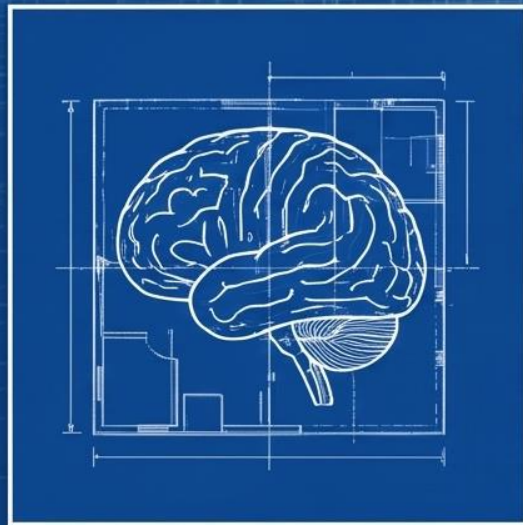
El desarrollo ya no es una marcha lineal hacia la deuda técnica. Es un ecosistema vivo donde las especificaciones nunca caducan y el código siempre se puede regenerar.

El Código se Pudre. El Conocimiento Perdura.



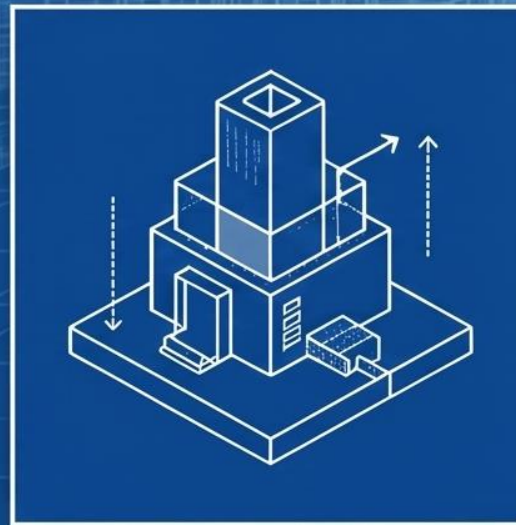
Tu Futuro no es la Sintaxis.

Es el diseño de sistemas y la gestión de agentes de IA.



Aduénate de la Intención.

El conocimiento del dominio y la arquitectura siempre serán responsabilidad humana.



Eleva el Nivel.

Asegura que el software que construyas hoy pueda ser regenerado para la tecnología de mañana.

